

```

action = input('What do you want to do? ')
action = action.lower() # force lowercase
action = action[0] # just use first letter
print()

if action == 'b':
    print('Get Balance:')
    ❸ userAccountNumber = input('Please enter your account number: ')
    userAccountNumber = int(userAccountNumber)
    userPassword = input('Please enter the password: ')
    theBalance = getBalance(userAccountNumber, userPassword)
    if theBalance is not None:
        print('Your balance is:', theBalance)

--- snipped additional user interface ---

print('Done')

```

Listing 1-5: Bank simulation with a parallel lists

At the beginning of the program, I set all three lists to the empty list ❶. To create a new account, I append the appropriate value to each of the three lists ❷.

Since I am now dealing with multiple accounts, I use the basic concept of a bank account number. Every time a user creates an account, the code uses the `len()` function on one of the lists and returns that number as the user's account number ❸, ❹. When I create an account for the first user, the length of the `accountNamesList` is zero. Therefore, the first account created will be given account number 0, the second account is given account number 1, and so on. Then, like at a real bank, to do any operation after creating an account (like deposit or withdraw funds), the user must supply their account number ❺.

However, this code is still working with global data; now there are three global lists of data.

Imagine viewing this data as a spreadsheet. It might look like [Table 1-1](#).